

MODULE 3

Python BDD Restful

Name	Chirashree Pan
Program / Year	B.Tech CSE (IoT, CS & BT) / 4th Year
Institution	Institute of Engineering & Management, Kolkata
Module Covered	Module 3 – Python BDD Restful
Enrollment No.	12023002029030
Experiments	Experiments 3.1 – 3.5
Date	19TH September , 2026

Index — Module 3 Experiments

Exp. No.	Experiment Title	Status
3.1	Python Basics for REST API Automation	Completed
3.2	Python HTTP Libraries – GET, POST, PATCH and DELETE	Completed
3.3	API Automation – Configuration, Sessions and Reusable Functions	Completed
3.4	Handling HTTP Responses – Status Codes, Headers and Exceptions	Completed
3.5	BDD with Python and Behave – REST API Scenario	Completed

Experiment 3.1 : Python Basics

1. Problem Statement

REST API automation needs basic Python concepts such as variables, dictionaries, functions and JSON data. The task is to use these concepts to read information from a REST API.

2. Objective

To understand the Python basics required for REST API automation and access values returned in JSON format.

3. Tools, Software & Concepts Used

- Python 3.x
- PyCharm
- Requests library
- JSON / dictionary data
- REST API

Concepts: variables, dictionaries, functions, JSON data, importing modules and basic output

4. Source Code / Implementation Steps

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

response = requests.get(url)
data = response.json()

print("Status Code:", response.status_code)
print("Post ID:", data["id"])
print("Title:", data["title"])
print("User ID:", data["userId"])
```

5. Output

5.1 TERMINAL OUTPUT



```
C:\Users\CAIRASHREE\AppData\Local\Programs\Python\Python314\Scripts>python .\main.py
Status Code: 200
Post ID: 1
Title: sed ut facere repellat provident occaecate magna accusamque etiam nihilque
User ID: 1
Program finished with exit code 0
```

Fig 3.1.1: Terminal output showing the execution result.

5.2 API RESPONSE

The REST API returned the requested post successfully with HTTP status code 200.

6. Result, Observation & Conclusion

Result:

The Python program was executed successfully, and a GET request was sent to the REST API. The API returned a successful response with status code **200**.

Observation:

The response received from the API was converted from JSON format into Python dictionary data. The required values such as Post ID, Title, and User ID were accessed using dictionary keys and displayed in the terminal.

Conclusion:

This experiment demonstrated how basic Python can be used for REST API automation. The terminal output confirmed that the API data was successfully received, processed, and displayed.

Experiment 3.2 : Python HTTP Libraries

1. Problem Statement

Python HTTP libraries are used to send requests to REST APIs. The task is to practise GET, POST, PATCH and DELETE requests and observe their response status codes.

2. Objective

To understand how the Requests library sends different HTTP methods and how payloads are passed to an API.

3. Tools, Software & Concepts Used

- Python 3.x
- PyCharm
- Requests library
- JSON
- REST API

Concepts: GET, POST with JSON payload, PATCH, DELETE, request data and response status codes

4. Source Code / Implementation Steps

```
import requests

url = "https://jsonplaceholder.typicode.com/posts"

response = requests.get(url + "/1")
print("GET Status:", response.status_code)
print("GET Data:", response.json())

payload = {
    "title": "Python Test",
    "body": "REST API automation",
    "userId": 1
}

response = requests.post(url, json=payload)
print("POST Status:", response.status_code)
print("POST Data:", response.json())

response = requests.patch(url + "/1", json={"title": "Updated Title"})
print("PATCH Status:", response.status_code)

response = requests.delete(url + "/1")
print("DELETE Status:", response.status_code)
```

5. Output

5.1 TERMINAL OUTPUT



```
"C:\Users\DHIRANJEE PARIKUPOTA\Local\Programs\Python\Python38\python.exe" "C:\Users\DHIRANJEE PARIKUPOTA\Projects\PythonProject\test.py"
GET Status: 200
GET Data: {'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expedit consectetur et nulla\nreprehenderit veniam deleniti quibusdam nostrum'}
POST Status: 201
POST Data: {'title': 'Python Test', 'body': 'REST API automation', 'userId': 1, 'id': 1011}
PATCH Status: 200
DELETE Status: 200

Process finished with exit code 0
```

Fig 3.2.1: Terminal output showing the execution result.

5.2 API RESPONSE

The REST API successfully processed GET, POST, PATCH, and DELETE requests and returned the corresponding HTTP status codes.

6. Result, Observation & Conclusion

Result:

The Python program was executed successfully, and GET, POST, PATCH, and DELETE requests were performed on the REST API.

Observation:

The GET request returned existing data with status code **200**. The POST request created a new record with status code **201**. The PATCH and DELETE requests were also completed successfully with status code **200**.

Conclusion:

This experiment demonstrated how Python's Requests library can be used to perform different HTTP operations. The terminal output confirmed that the API requests were sent successfully and the expected response data and status codes were received.

Experiment 3.3 : API Automation

1. Problem Statement

Writing every API request separately can make an automation script repetitive. The task is to use a common base URL, reusable functions and a session for API automation.

2. Objective

To create a simple reusable Python API automation program using functions and a Requests Session.

3. Tools, Software & Concepts Used

- Python 3.x
- PyCharm
- Requests library
- REST API
- Python functions
- Requests Session

Concepts: global configuration, reusable functions, session management, API methods and JSON data

4. Source Code / Implementation Steps

```
import requests

BASE_URL = "https://jsonplaceholder.typicode.com"
session = requests.Session()

def get_post(post_id):
    response = session.get(f'{BASE_URL}/posts/{post_id}')
    print("Status:", response.status_code)
    print("Title:", response.json()["title"])

def create_post():
    payload = {
        "title": "Automation Test",
        "body": "Testing API using Python",
        "userId": 1
    }

    response = session.post(f'{BASE_URL}/posts', json=payload)
    print("Create Status:", response.status_code)
    print("Response:", response.json())

get_post(1)
create_post()

session.close()
print("API session closed")
```

5. Output

5.1 TERMINAL OUTPUT

A terminal window with a dark background and light-colored text. The text shows the execution of a Python script. It starts with the command prompt, followed by the status code 200, a title, the create status 201, a JSON response, and finally the API session closed and process finished messages.

```
"C:\Users\CHIRASHREE PAN\AppData\Local\Programs\Python\Python14\python.exe" "C:\Users\CHIRASHREE PAN\PycharmProjects\PythonProject\test.py"
Status: 200
Title: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
Create Status: 201
Response: {'title': 'Automation Test', 'body': 'Testing API using Python', 'userId': 1, 'id': 101}
API session closed

Process finished with exit code 0
```

Fig 3.3.1: Terminal output showing the execution result.

5.2 API RESPONSE

The API successfully returned the requested post and created a new record using the POST request.

6. Result, Observation & Conclusion

Result:

The Python API automation program was executed successfully. The GET request retrieved the required post, and the POST request created a new record successfully.

Observation:

The GET request returned status code **200** and displayed the post title. The POST request returned status code **201** and displayed the newly created record. The API session was then closed properly.

Conclusion:

This experiment demonstrated how reusable Python functions and a Requests Session can be used for API automation. The terminal output confirmed that the API requests were processed successfully and the expected response data was received.

Experiment 3.4 : Handling HTTP Responses

1. Problem Statement

REST API tests need to check status codes and response content and also handle request errors. The task is to read the response object and use exception handling.

2. Objective

To understand response objects, status codes, headers, JSON content and exception handling in Python API automation.

3. Tools, Software & Concepts Used

- Python 3.x
- PyCharm
- Requests library
- HTTP status codes
- Exception handling

Concepts: response object, status codes, response headers, response text/JSON, timeout and exceptions

4. Source Code / Implementation Steps

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

try:
    response = requests.get(url, timeout=10)

    print("Status Code:", response.status_code)
    print("Content Type:", response.headers.get("Content-Type"))

    if response.ok:
        data = response.json()
        print("Response ID:", data["id"])
        print("Response Text:", data["body"])
    else:
        print("Request was not successful")

except requests.exceptions.RequestException as error:
    print("Request Error:", error)
```

5. Output

5.1 TERMINAL OUTPUT



```
"C:\Users\CHIRASHREE PAN\AppData\Local\Programs\Python\Python314\python.exe" "C:\Users\CHIRASHREE PAN\PycharmProjects\PythonProject\test.py"
Status Code: 200
Content Type: application/json; charset=utf-8
Response ID: 1
Response Text: quia et suscipit
                suscipit recusandae consequuntur expedita et cum
                reprehenderit molestiae ut ut quas totam
                nostrum rerum est autem sunt rem eveniet architecto
Process finished with exit code 0
```

Fig 3.4.1: Terminal output showing the execution result.

5.2 API RESPONSE

The API returned a successful JSON response with status code 200, and the required response fields were displayed

6. Result, Observation & Conclusion

Result:

The Python program was executed successfully and received a response from the REST API with status code **200**.

Observation:

The response content type was identified as JSON. The program successfully accessed the response ID and response text from the returned JSON data.

Conclusion:

This experiment demonstrated how HTTP responses can be handled and checked using Python. The terminal output confirmed that the request was successful and that the required response information was received and processed correctly.

Experiment 3.5 : BDD

1. Problem Statement

Behaviour Driven Development allows a test scenario to be written in readable Given, When and Then steps. The task is to connect a Behave feature file with Python code that calls a REST API.

2. Objective

To understand the basic structure of a Behave BDD API test and connect readable Gherkin steps with Python step definitions.

3. Tools, Software & Concepts Used

- Python 3.x
- PyCharm
- Behave
- Requests library
- Gherkin syntax
- REST API

Concepts: BDD, feature file, scenario, Given/When/Then, step definitions and API validation

4. Source Code / Implementation Steps

```
#features/api_test.feature

Feature: REST API testing

  Scenario: Get a post from REST API
    Given the REST API is available
    When I send a GET request for post 1
    Then the response status should be 200

# features/steps/api_steps.py

import requests
from behave import given, when, then

@given("the REST API is available")
def step_api_available(context):
    context.url = "https://jsonplaceholder.typicode.com"

@when("I send a GET request for post 1")
def step_send_request(context):
    context.response = requests.get(context.url + "/posts/1")

@then("the response status should be 200")
def step_check_status(context):
    assert context.response.status_code == 200
    print("API test passed")
```

5. Output

5.1 TERMINAL OUTPUT

A terminal window showing the execution of a Behave BDD test. The prompt is 'PS C:\Users\Vidya3a\Proj\Project\PythonProject> cd Mod\'. The command executed is 'PS C:\Users\Vidya3a\Proj\Project\PythonProject\Mod > .\venv\Scripts\python.exe .\test1.py'. The output shows 'USING RUNNER: DefaultRunner', 'Features/ REST API testing | FeatureTest/Feature', and a scenario: 'Scenario: Get a post from REST API Given the REST API is available When I send a GET request for post 1 Then the response status should be 200 @ Features/api_test.feature:4'. The results are: '1 feature: passed, 0 failed, 0 skipped', '1 scenario: passed, 0 failed, 0 skipped', '1 step passed, 0 failed, 0 skipped', and 'Took 0m0.00s'.

```
PS C:\Users\Vidya3a\Proj\Project\PythonProject> cd Mod\
PS C:\Users\Vidya3a\Proj\Project\PythonProject\Mod > .\venv\Scripts\python.exe .\test1.py
USING RUNNER: DefaultRunner
Features/ REST API testing | FeatureTest/Feature
Scenario: Get a post from REST API Given the REST API is available When I send a GET request for post 1 Then the response status should be 200 @ Features/api_test.feature:4
1 feature: passed, 0 failed, 0 skipped
1 scenario: passed, 0 failed, 0 skipped
1 step passed, 0 failed, 0 skipped
Took 0m0.00s
```

Fig 3.5.1: Terminal output showing the execution result.

5.2 BDD OUTPUT

The Behave scenario was executed successfully. The GET request returned status code 200 and all three BDD steps passed.

6. Result, Observation & Conclusion

Result:

The Behave BDD test was executed successfully, and the REST API scenario passed without any errors.

Observation:

The Given step confirmed that the REST API was available. The When step sent a GET request for post 1, and the Then step verified that the response status code was 200. All three steps passed successfully.

Conclusion:

This experiment demonstrated the use of BDD with Python and Behave for REST API testing. The feature file described the test scenario using Given, When and Then statements, while Python step definitions performed the actual API operations. The terminal output confirmed that the feature, scenario and all three steps were successful.